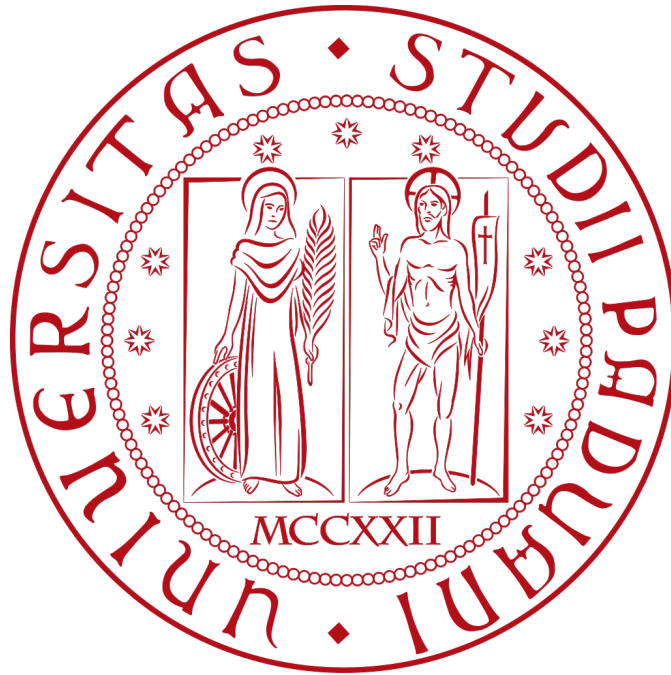




ZUCCHETTI



AI TrAIning

Specifica tecnica

Filippo Sabbadin

08 / 08 / 2025

Indice

1. Introduzione	1
1.1. Scopo del documento	1
1.2. Scopo del prodotto	1
1.3. Glossario	2
1.4. Riferimenti	2
1.4.1. Riferimenti normativi	2
1.4.2. Riferimenti informativi	2
1.4.3. Riferimenti tecnici	3
2. Tecnologie	4
3. Premessa	5
3.1. Concetti chiave di Godot	5
3.1.1. Nodi	5
3.1.2. Scene	6
3.1.3. Segnali	6
3.2. Limitazioni di GDScript	7
4. Architettura	9
4.1. Introduzione	9
4.1.1. Funzioni comuni	9
4.2. Asset comuni	10
4.2.1. Giocatore	10
4.2.1.1. CameraRayCast	11
4.2.1.2. StateMachine	12
4.2.1.3. PlayerSavesHandler	13
4.2.1.4. Movement	13
4.2.1.5. GrabItem	13
4.2.1.6. ParticleEmitter	13
4.2.1.7. PlayerUI	13
4.2.2. Interazione	13
4.2.2.1. InteractableArea	14
4.2.2.2. NPC	14
4.2.2.3. InteractableSign	14
4.2.2.4. NPCDialogue	14
4.2.3. Dialoghi	14
4.2.3.1. Dialogue	15
4.2.3.2. DialogueBoxSimple	15
4.2.3.3. DialogueBoxOptions	15
4.2.3.4. DialogueOptionsButtons	15

4.2.4. Salvataggi	15
4.2.5. Singletons / Autoloads	16
4.3. Struttura base livello	16
4.4. Livello «Regressione lineare»	17
4.4.1. Cannone e grafico LR	17
4.5. Livello «Albero di decisione»	17
4.5.1. Albero	17
4.6. Livello «Causalità»	17
4.6.1. Scena di intermezzo	17
5. Requisiti soddisfatti	18
5.1. Tabella requisiti soddisfatti	18
5.2. Grafico requisiti soddisfatti	18

Lista di immagini

Figura 1	Scena del giocatore	6
Figura 2	Diagramma delle classi del giocatore	10
Figura 3	Diagramma delle classi della telecamera del giocatore	11
Figura 4	Diagramma sulla struttura della macchina di stati	12
Figura 5	Diagramma degli oggetti con cui il giocatore può interagire	13
Figura 6	Diagramma sul funzionamento di un dialogo	14
Figura 7	Diagramma sul funzionamento dei salvataggi	15
Figura 8	Classi <i>Autoloads</i>	16
Figura 9	Diagramma di un livello base	16
Figura 10	Diagramma del livello della causalità	17

Lista di tabelle

Tabella 1	Tecnologie utilizzate	4
------------------	------------------------------------	----------

1. Introduzione

1.1. Scopo del documento

Lo scopo di questo documento è descrivere l'architettura e le scelte relative a essa che sono state fatte durante la fase di progettazione e codifica del progetto.

Vengono riportati i grafici UML delle classi per rappresentare l'architettura finale dell'applicazione.

1.2. Scopo del prodotto

Il progetto consiste nello sviluppo di un videogioco con il motore di gioco [Godot*](#) di tipo [platformer*](#), in cui il giocatore controlla un personaggio che si muove in un ambiente tridimensionale. Il gioco si basa su temi di [Intelligenza Artificiale*](#) e [Machine Learning*](#), sviluppando meccaniche ed elementi del livello basati su questi argomenti.

Comprende 3 livelli, ognuno con un tema diverso, un menu principale che viene visualizzato non appena il gioco viene avviato, e un menu di pausa che può essere visualizzato in qualsiasi momento durante il gioco.

Ogni livello presenta meccaniche diverse e uniche.

- **Regressione lineare**

Il livello presenta una serie di grafici con una linea e dei dati, accedendo ad un «cannone» il giocatore può posizionare dei nuovi dati nel grafico e la retta si modifica in base alla posizione del nuovo punto. Nel caso i punti siano stati piazzati male e non sia possibile modificare ulteriormente la linea, il giocatore può resettare il grafico premendo il tasto apposito.

Se soddisfatto della rotazione e posizione della linea il giocatore può uscire dal cannone e camminare sopra la linea per proseguire nel livello.

- **Albero di decisione**

In questo livello sono presenti diversi cani, ognuno di una razza diversa, ed un albero di decisione tridimensionale. Ad ogni nodo dell'albero viene mostrata una domanda e la direzione da seguire per ogni risposta.

L'obiettivo del giocatore è rispondere correttamente a ognuna di queste domande e posizionare il cane nel nodo finale giusto. In caso corretto, il giocatore potrà visualizzare la razza indovinata in un tabellone, e questo darà un Training Data al giocatore per ogni razza indovinata.

Infine è presente un NPC che permette al giocatore di riportare tutti i cani nella posizione iniziale nel caso siano troppo sparsi.

- **Causalità**

Appena caricato nel livello, il giocatore avrà la possibilità di parlare con un NPC gelataio che lo guiderà all'obiettivo di questo livello, cioè accendere tutte le unità di condizionatori esterne presenti, poiché crede che le vendite di gelati e l'uso dei condizionatori siano correlate. Nel livello sono presenti anche dei grafici che cambiano durante il livello in base alle azioni del giocatore.

Una volta accese tutte le unità, verrà avviata una [scena di intermezzo*](#) e alla fine di essa il giocatore dovrà indicare la vera causa della vendita di gelati.

1.3. Glossario

Per facilitare la comprensione del documento, è stato creato un glossario che contiene i termini utilizzati nel documento e le loro definizioni. I termini presenti nel glossario sono colorati di blu e seguiti da un'asterisco: [esempio*](#).

Il glossario è accessibile tramite il link:

<https://github.com/fliposab/ProgettoStage/blob/main/Documentazione/Glossario.pdf>

oppure consultando il rispettivo documento all'interno della stessa cartella.

1.4. Riferimenti

1.4.1. Riferimenti normativi

- Piano di lavoro:

<https://github.com/fliposab/ProgettoStage/blob/main/Documentazione/Piano-di-lavoro.pdf>

- Norme di progetto:

<https://github.com/fliposab/ProgettoStage/blob/main/Documentazione/Norme-di-progetto.pdf>

1.4.2. Riferimenti informativi

- Slide T05 del corso di Ingegneria del Software:

<https://www.math.unipd.it/~tullio/IS-1/2024/Dispense/T05.pdf>

- Diagrammi UML - Use case:

da cambiare

-
- Documentazione «Godot Engine»:

<https://docs.godotengine.org/en/stable/>

1.4.3. Riferimenti tecnici

- Documentazione di Godot:

<https://docs.godotengine.org/it/4.x/about/introduction.html>

- I concetti chiave di Godot:

https://docs.godotengine.org/it/4.x/getting_started/introduction/key_concepts_overview.html

- La filosofia progettuale di Godot:

https://docs.godotengine.org/it/4.x/getting_started/introduction/godot_design_philosophy.html

- Architettura di Godot:

https://docs.godotengine.org/en/stable/contributing/development/core_and_modules/godot_architecture_diagram.html

2. Tecnologie

Nome	Descrizione	Versione
Codice		
GDScript		(Legata a Godot)
GDSshader		(Legata a Godot)
Typst	Linguaggio utilizzato per la stesura dei documenti	
Softwares		
Godot	Il motore di gioco open source per lo sviluppo del videogioco.	4.5-beta3-mono
Blender	Software di modellazione ed animazione 3D usato per creare i modelli 3D del gioco	4.4.3
GIMP	Software di modifica di immagini, usato per modificare le textures del gioco	3.0.4
Strumenti e servizi		
Git		2.50.1
GitHub Actions	Servizio di integrazione continua e distribuzione continua (CI/CD), utilizzato per compilare i documenti ad ogni push	-
Tipi di files non generati dagli strumenti elencati sopra		
*.csv	«Comma separated values», file utilizzato per memorizzare le frasi nelle lingue diverse supportate dal gioco	-
*.ini	Tipo di file «plain-text» utilizzato per salvare i dati del gioco	-
*.glb	«GLTF Binary», file utilizzato per memorizzare i modelli 3D e le loro animazioni in formato binario, in modo da risparmiare spazio e migliorare le prestazioni	2.0.1

Tabella 1: Tecnologie utilizzate

3. Premessa

Prima di iniziare a descrivere l'architettura del progetto, è necessario introdurre alcuni concetti chiave di Godot e le limitazioni del linguaggio GDScript, che è il linguaggio di programmazione principale utilizzato in Godot.

3.1. Concetti chiave di Godot

3.1.1. Nodi

Dalla documentazione di Godot:

I nodi sono i blocchi fondamentali del gioco. Sono come ingredienti in una ricetta. Ci sono dozzine di tipi che possono mostrare un'immagine, riprodurre un suono, rappresentare una camera, e molto altro. Tutti i nodi hanno le seguenti caratteristiche:

- *Un nome.*
- *Proprietà modificabili.*
- *Ricevono callback per aggiornarsi ad ogni frame.*
- *Si possono estendere con nuove proprietà e funzioni.*
- *Si possono aggiungere a un altro nodo come figlio.*

Inoltre ad ogni nodo si può assegnare uno script, che estende il tipo di quel nodo e aggiunge nuove funzionalità.

I principali tipi di nodi che vengono utilizzati in questo progetto sono:

- **Node**: nodo base da cui vengono estesi tutti gli altri nodi, in questo progetto viene usato per assegnare classi e inserirli come figli in altri nodi.
- **Node3D**: rappresenta un oggetto nello spazio tridimensionale.
 - **CharacterBody3D**: rappresenta un personaggio nel gioco, gestendo la sua posizione, animazione e interazioni.
 - **Camera3D**: rappresenta una telecamera nello spazio tridimensionale, che può essere utilizzata per visualizzare la scena.
 - **MeshInstance3D**: rappresenta un oggetto tridimensionale con una mesh, che può essere utilizzato per visualizzare modelli 3D.
 - **CollisionShape3D**: rappresenta una forma di collisione nello spazio tridimensionale, utilizzata per gestire le interazioni fisiche tra gli oggetti.
 - **Area3D**: rappresenta un'area nello spazio tridimensionale, utilizzata per gestire le interazioni tra gli oggetti all'interno di essa.
- **AnimationPlayer**: gestisce le animazioni degli oggetti nella scena, permettendo di riprodurre animazioni su mesh, telecamere e altri nodi.
- **Control**: rappresenta un nodo di interfaccia utente, utilizzato per gestire gli elementi dell'interfaccia grafica del gioco.

3.1.2. Scene

Dalla documentazione di Godot:

Quando organizzi nodi in un albero, come il nostro personaggio, possiamo chiamare questa formazione una scena. Una volta salvata, la scena si presenta come un nuovo nodo nell'editor, dove possiamo aggiungerlo come figlio di un nodo esistente. In questo caso, l'istanza della scena appare come nodo singolo con interni nascosti. Le scene consentono di strutturare il codice del gioco in qualunque modo tu voglia. Puoi comporre nodi per creare nodi personalizzati e complessi, come un personaggio di gioco che si muove e salta, una barra della vita, una cesta con cui puoi interagire, e molto altro.

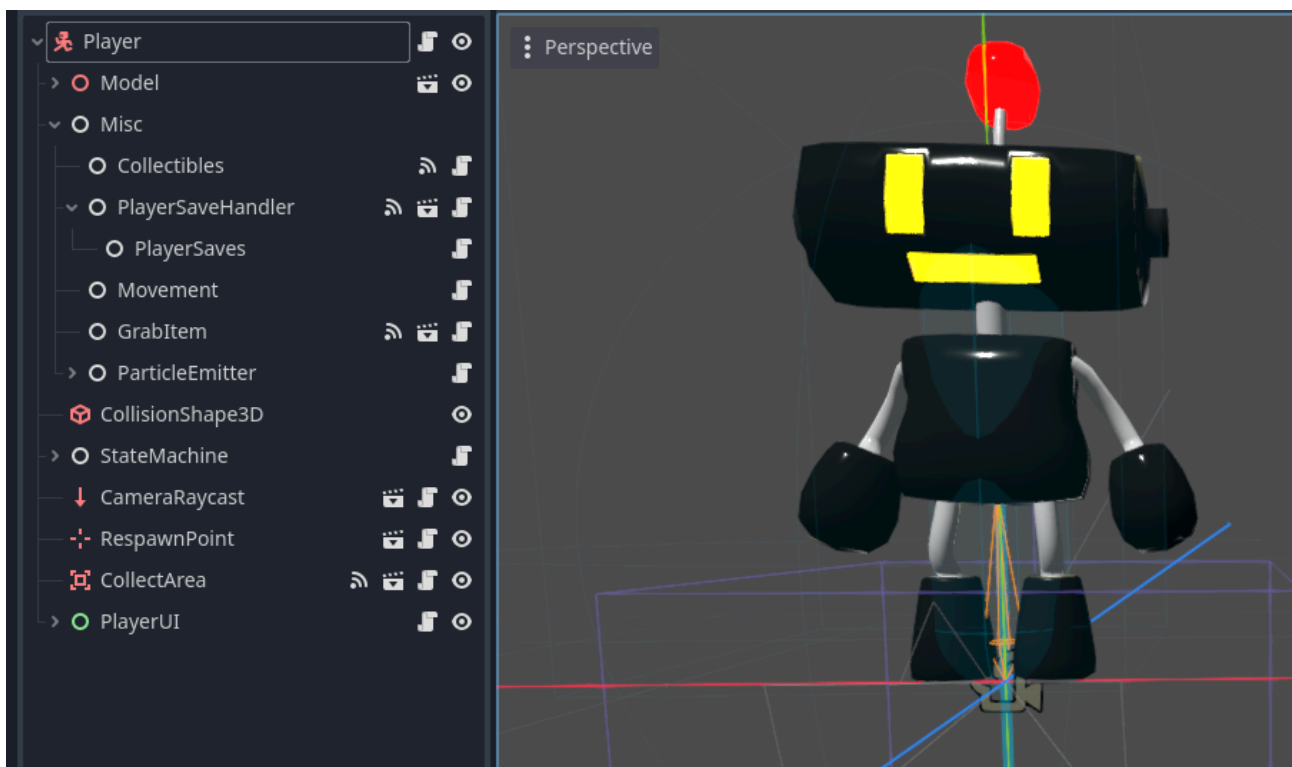


Figura 1: Scena del giocatore

Oltre che a comportarsi come nodi, le scene hanno anche le seguenti caratteristiche:

- Hanno sempre un nodo *owner* come il «Player» nel nostro esempio.
- Si possono salvare sul disco locale e caricarle in seguito.
- Si possono creare quante più istanze di una scena si desidera. Ad esempio, si possono avere cinque o dieci personaggi nel gioco, creati da una determinata scena.

3.1.3. Segnali

I segnali sono un modo per far comunicare i nodi in maniera asincrona in Godot. Ogni classe presenta dei segnali preimpostati ed emessi in determinati momenti, ad esempio quando un

nodo viene caricato, questo emette il segnale *ready*, oppure quando un bottone viene premuto, viene emesso il segnale *pressed*.

I segnali inoltre possono anche contenere dei parametri, che possono essere utilizzati per passare informazioni tra i nodi.

Infine, si possono creare segnali personalizzati, che possono essere emessi in qualsiasi momento dal nodo che li ha definiti tramite il metodo *signal.emit(...)*.

Ci sono due modi per collegare un segnale ad un altro nodo:

- tramite l'editor: selezionando il nodo che emette il segnale e trascinandolo sul nodo che deve ricevere il segnale, e selezionando il metodo che deve essere chiamato quando il segnale viene emesso;
- tramite codice: utilizzando il metodo *signal.connect(Callable)* del nodo che emette il segnale, passando come parametro il nome del segnale e il nodo che deve ricevere il segnale, e il metodo che deve essere chiamato quando il segnale viene emesso.

3.2. Limitazioni di GDScript

GDScript è un linguaggio di programmazione che, sebbene sia molto simile a Python, ha alcune limitazioni rispetto ad esso.

• Variabili private

In GDScript non è possibile definire variabili private, ma è possibile utilizzare la convenzione di denominazione con il trattino basso iniziale per indicare che una variabile non dovrebbe essere accessibile al di fuori della classe.

Nei diagrammi presentati in questo documento, le variabili segnate come private in realtà sono le variabili precedute da un trattino basso, ma trattate comunque come variabili private per garantire compatibilità con versioni future di GDScript nel caso vengano implementate variabili private.

• Funzioni virtuali

GDScript non presenta la possibilità di dichiarare esplicitamente una funzione come virtual, ad esempio tramite la parola chiave *virtual* come in C#. Tuttavia queste funzioni possono comunque essere sovrascritte dalle funzioni presenti nelle classi derivate.

• Assenza di interfacce

Tra queste limitazioni vi è l'assenza di interfacce, che sono un costrutto di programmazione che permette di definire un contratto che le classi devono rispettare, specificando i metodi che devono essere implementati.

Per ovviare a questa mancanza, GDScript permette di usare [duck-typing*](#), che consente di verificare il tipo di un oggetto in fase di esecuzione, piuttosto che in fase di compilazione.

Un altro modo per fornire i metodi che una classe deve implementare è utilizzare classi astratte ed ereditarietà.

4. Architettura

4.1. Introduzione

Di seguito viene presentata l'architettura del progetto tramite diagrammi UML delle classi, che mostrano le relazioni tra le classi utilizzate nel progetto.

L'applicazione è strutturata come un monolite, decisione presa per i seguenti motivi:

- **semplicità:** l'applicazione è relativamente piccola e non richiede una struttura complessa per essere gestita;
- **facilità di manutenzione:** la struttura monolitica permette di gestire più facilmente le dipendenze tra le classi, poiché tutte le classi sono contenute in un unico file e non è necessario gestire le dipendenze tra moduli diversi;
- **migliori prestazioni:** in un videogioco le prestazioni sono fondamentali e una struttura monolitica può contribuire a ottimizzare le performance, riducendo i tempi di caricamento e migliorando la fluidità del gioco.

4.1.1. Funzioni comuni

Molte classi del progetto presentano delle funzioni virtuali comuni, che vengono fornite dalle classi base del motore di gioco:

+ready(): void

Questa funzione viene chiamata quando il nodo è pronto per essere utilizzato, ovvero quando tutti i nodi figli sono stati caricati e il nodo è pronto per essere utilizzato.

In questa funzione è possibile inizializzare le variabili, collegare i segnali e impostare le proprietà del nodo.

È importante notare che questa funzione viene chiamata solo una volta, quando il nodo viene caricato per la prima volta nella scena, e non ad ogni frame del gioco.

+process(delta: float): void

Questa funzione viene chiamata ad ogni frame del gioco e permette di aggiornare lo stato della classe ad ogni frame. Il parametro *delta* rappresenta il tempo trascorso dall'ultimo frame, ed è utile per gestire le animazioni e le interazioni in modo fluido e coerente.

+physics_process(delta: float): void

Questa funzione viene chiamata ad ogni frame di fisica del gioco, che di default è fisso 60 volte al secondo, anche se il numero di frame al secondo del gioco sia inferiore.

Questa funzione è utile per gestire le interazioni fisiche tra gli oggetti, come ad esempio la gestione delle collisioni e la gestione della gravità.

Il parametro *delta* rappresenta il tempo trascorso dall'ultimo frame di fisica.

+input(event: InputEvent): void

Funzione chiamata ogni volta che il gioco rileva un qualsiasi input, sia da tastiera che da joystick. Il parametro *event* rappresenta l'input che chiama la funzione.

4.2. Asset comuni

Di seguito viene mostrata l'architettura degli [asset*](#) presenti in più parti del gioco che non sono unici ad un livello specifico.

4.2.1. Giocatore

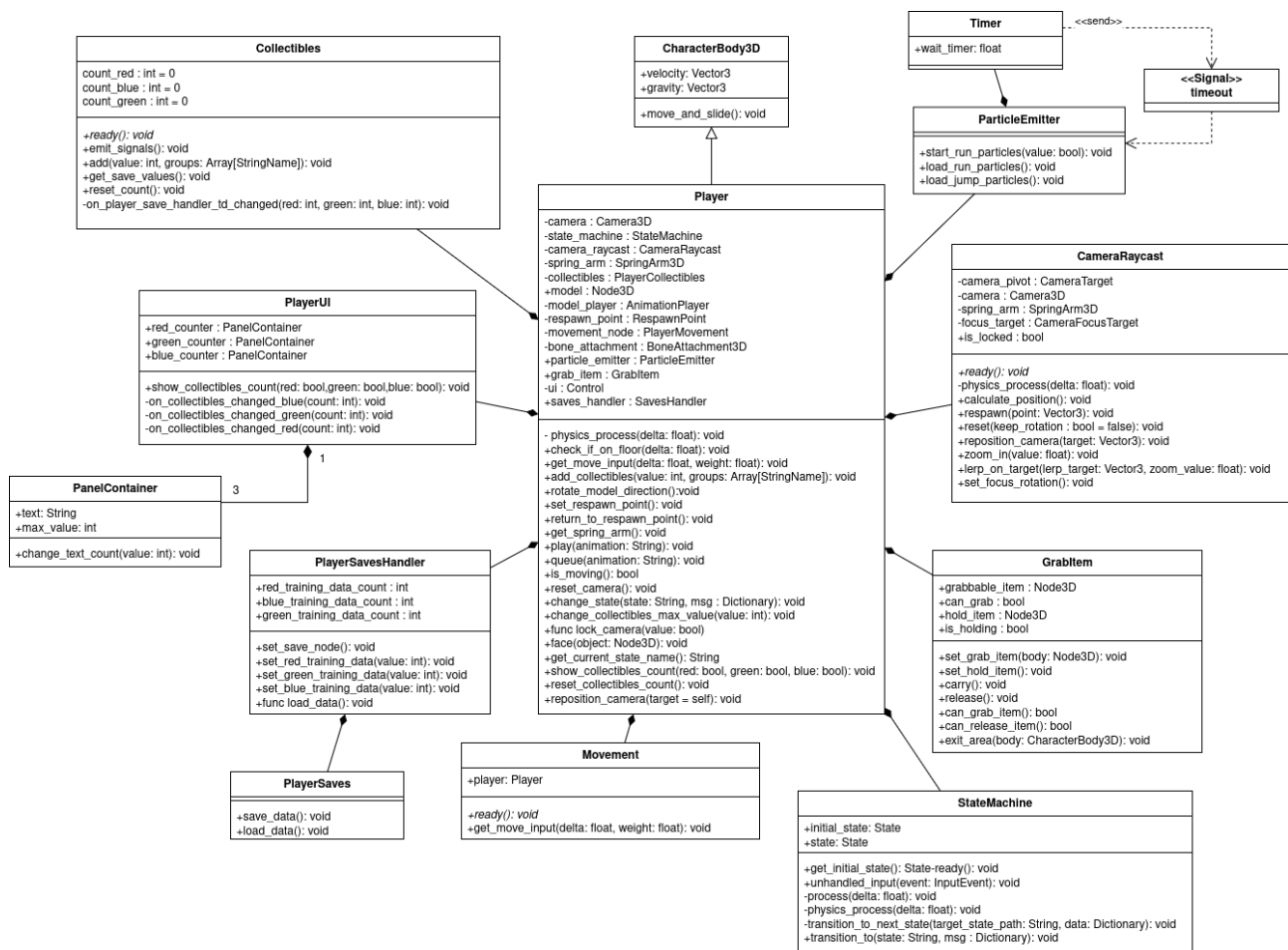


Figura 2: Diagramma delle classi del giocatore

Il giocatore può essere considerata la classe principale di tutta l'applicazione, attraverso il quale l'utente può interagire con la maggior parte dell'applicazione.

Nonostante ci sia solo un giocatore presente del gioco, questo non è un [singleton*](#), poiché per implementare un singleton in Godot è richiesto che questo sia caricato come [autoload*](#) in ogni scena, e non c'è motivo di caricare il giocatore nel menu principale, all'avvio del gioco.

Molte variabili presenti nel giocatore sono riferimenti ai suoi nodi figli presenti nella scena, queste variabili sono precedute dalla parola chiave *@onready* nel codice. Similmente, molte funzioni della classe servono solo per accedere alle variabili dei suoi nodi figli.

La classe del giocatore ha associate le seguenti classi divise per funzionalità:

- **CameraRaycast**
- **StateMachine**
- **PlayerSavesHandler**
- **Movement**
- **GrabItem**
- **ParticleEmitter**
- **PlayerUI**

4.2.1.1. CameraRayCast

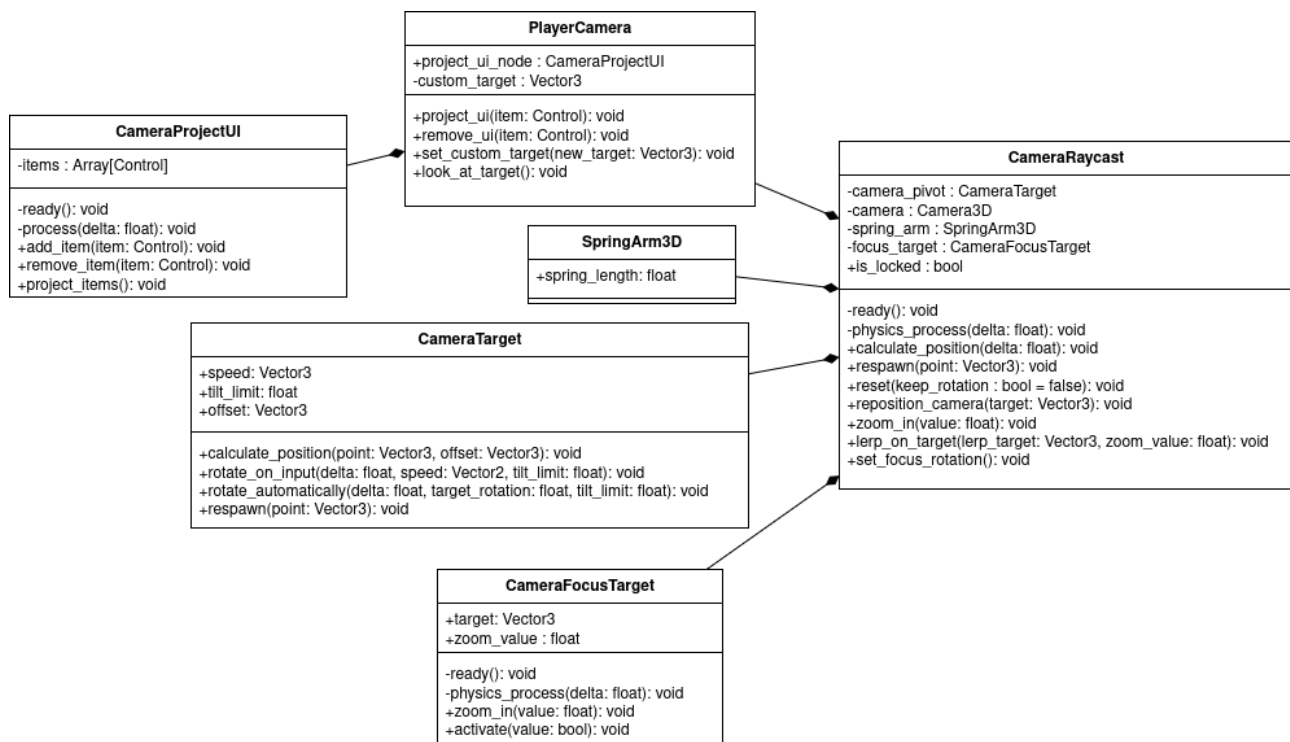


Figura 3: Diagramma delle classi della telecamera del giocatore

4.2.1.2. StateMachine

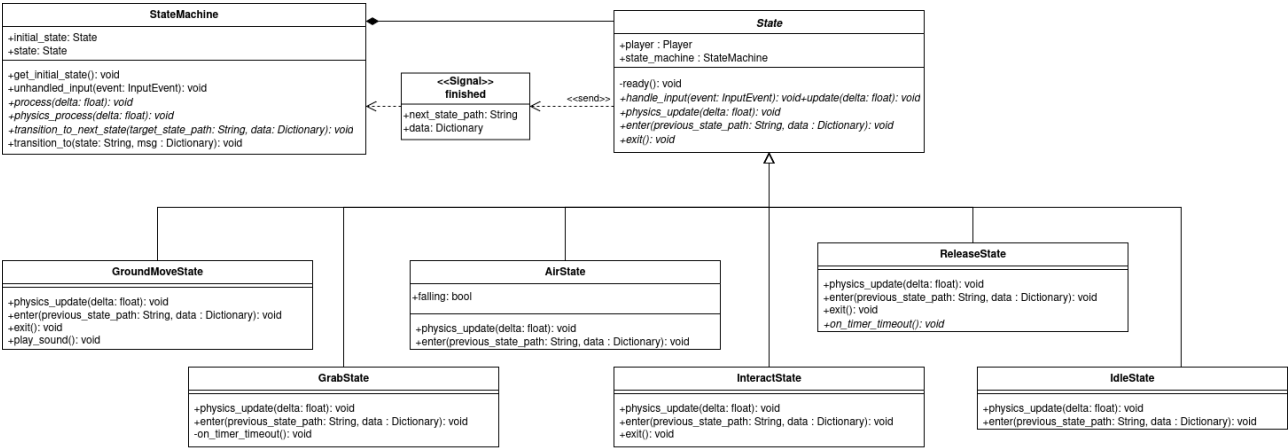


Figura 4: Diagramma sulla struttura della macchina di stati

4.2.1.3. PlayerSavesHandler

4.2.1.4. Movement

4.2.1.5. GrabItem

4.2.1.6. ParticleEmitter

4.2.1.7. PlayerUI

4.2.2. Interazione

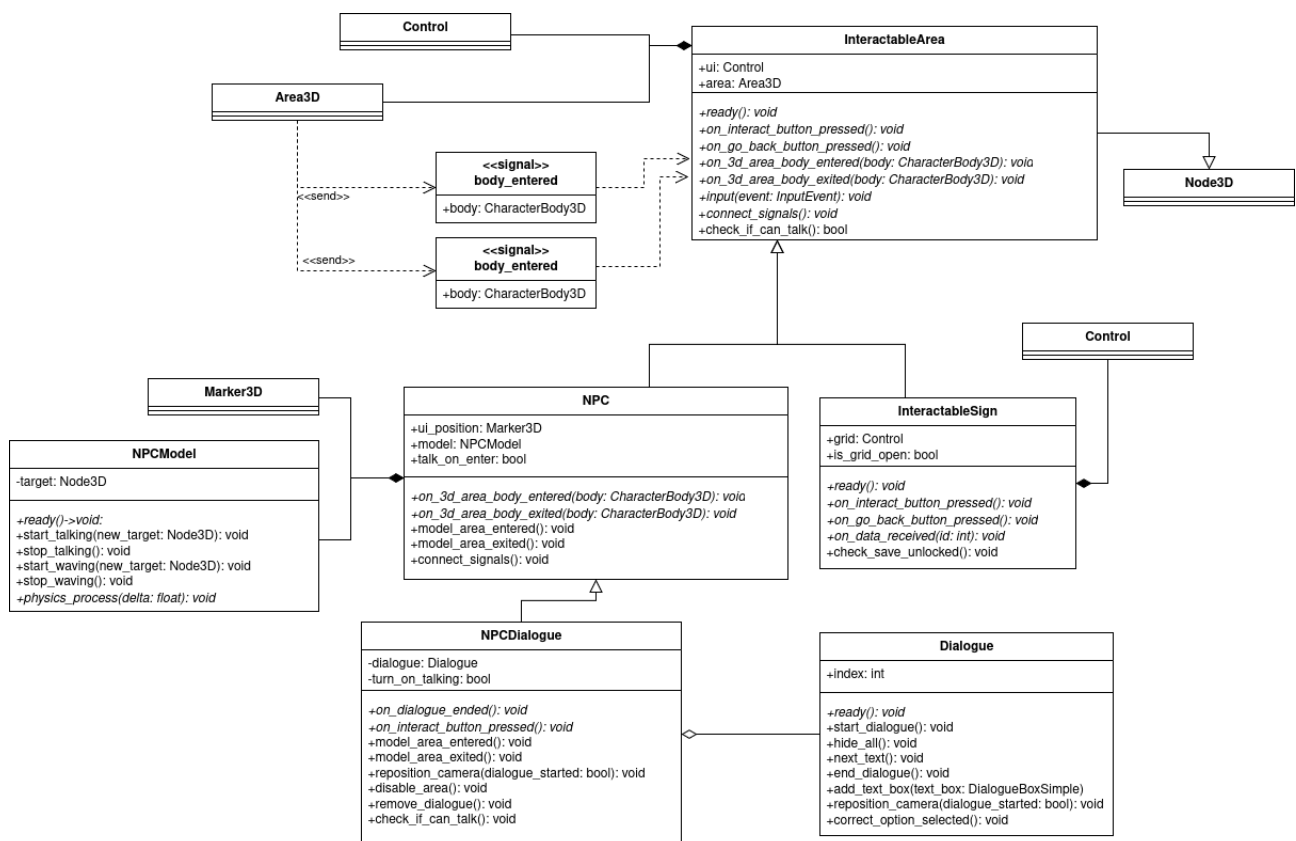


Figura 5: Diagramma degli oggetti con cui il giocatore può interagire

4.2.2.1. InteractableArea

4.2.2.2. NPC

4.2.2.3. InteractableSign

4.2.2.4. NPCDialogue

4.2.3. Dialoghi

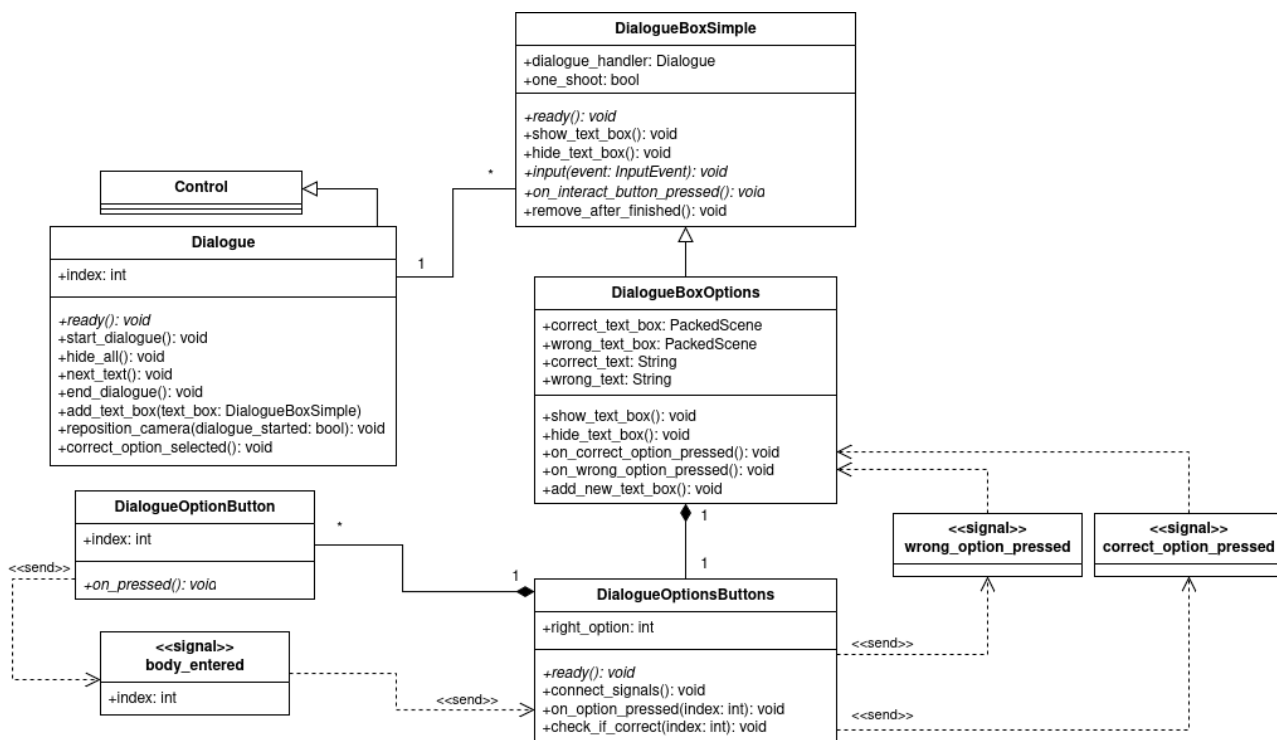


Figura 6: Diagramma sul funzionamento di un dialogo

4.2.3.1. Dialogue

4.2.3.2. DialogueBoxSimple

4.2.3.3. DialogueBoxOptions

4.2.3.4. DialogueOptionsButtons

4.2.4. Salvataggi

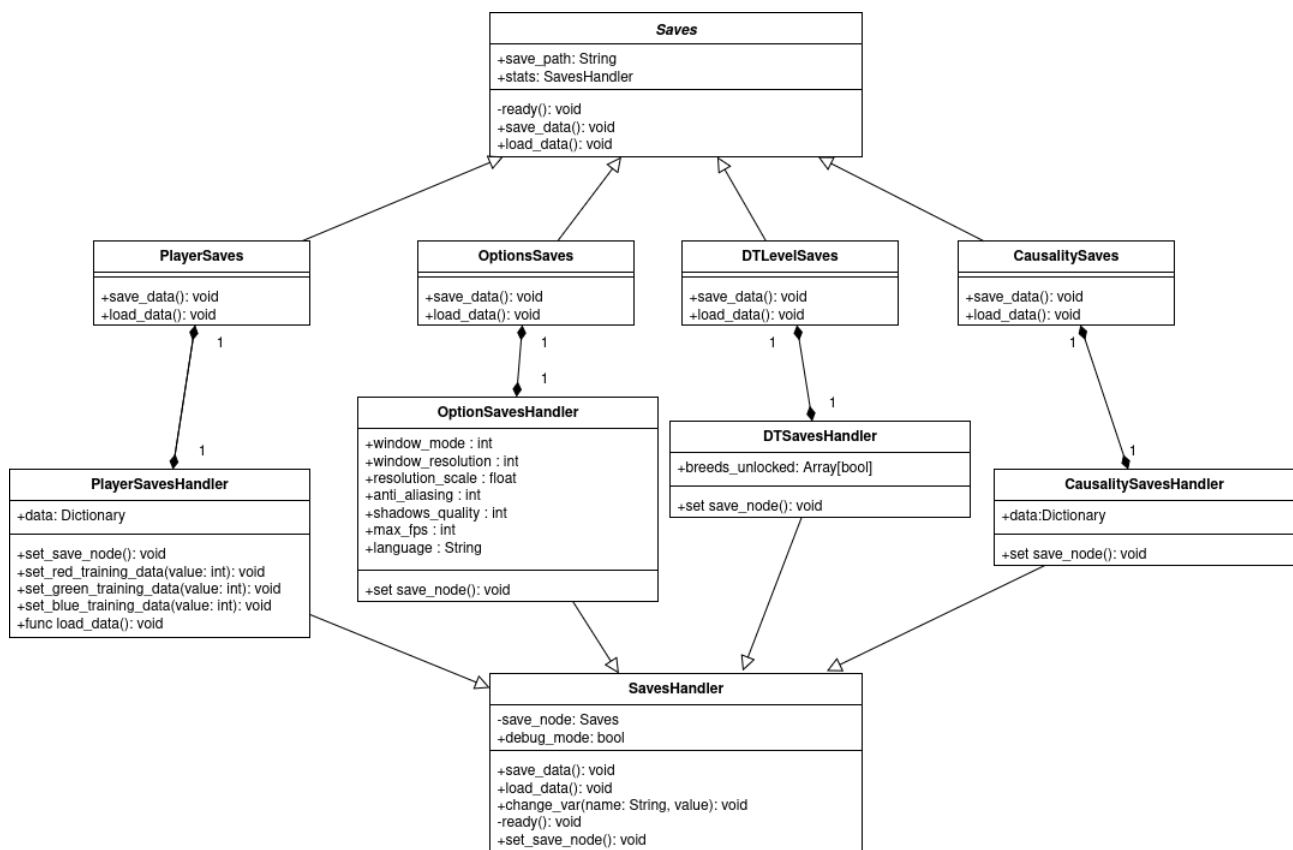


Figura 7: Diagramma sul funzionamento dei salvataggi

4.2.5. Singletons / Autoloads

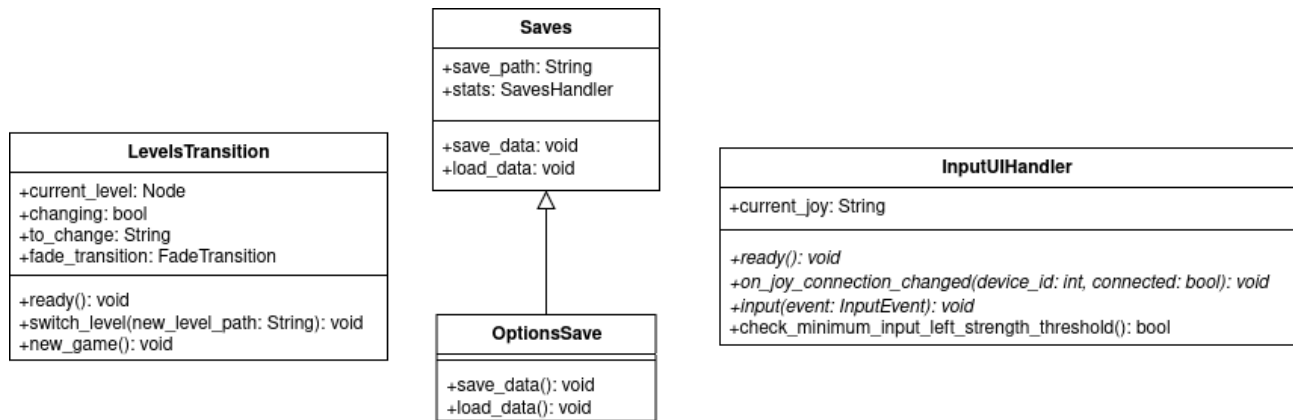


Figura 8: Classi Autoloads

4.3. Struttura base livello

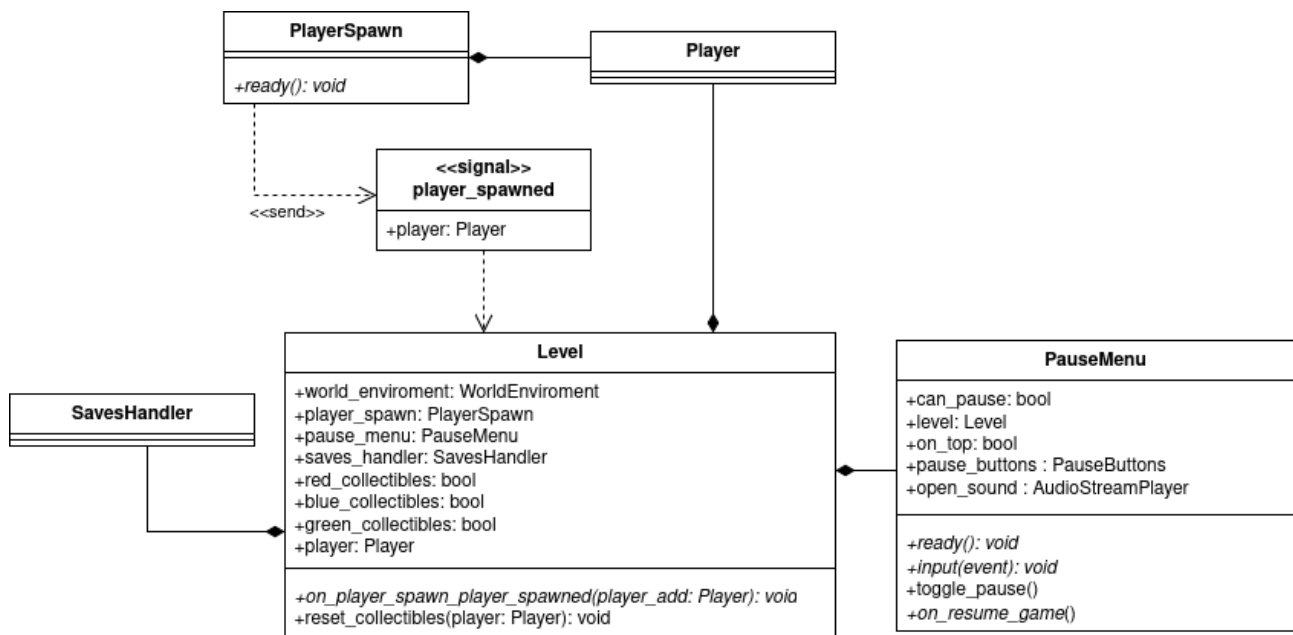


Figura 9: Diagramma di un livello base

4.4. Livello «Regressione lineare»

4.4.1. Cannone e grafico LR

4.5. Livello «Albero di decisione»

4.5.1. Albero

4.6. Livello «Causalità»

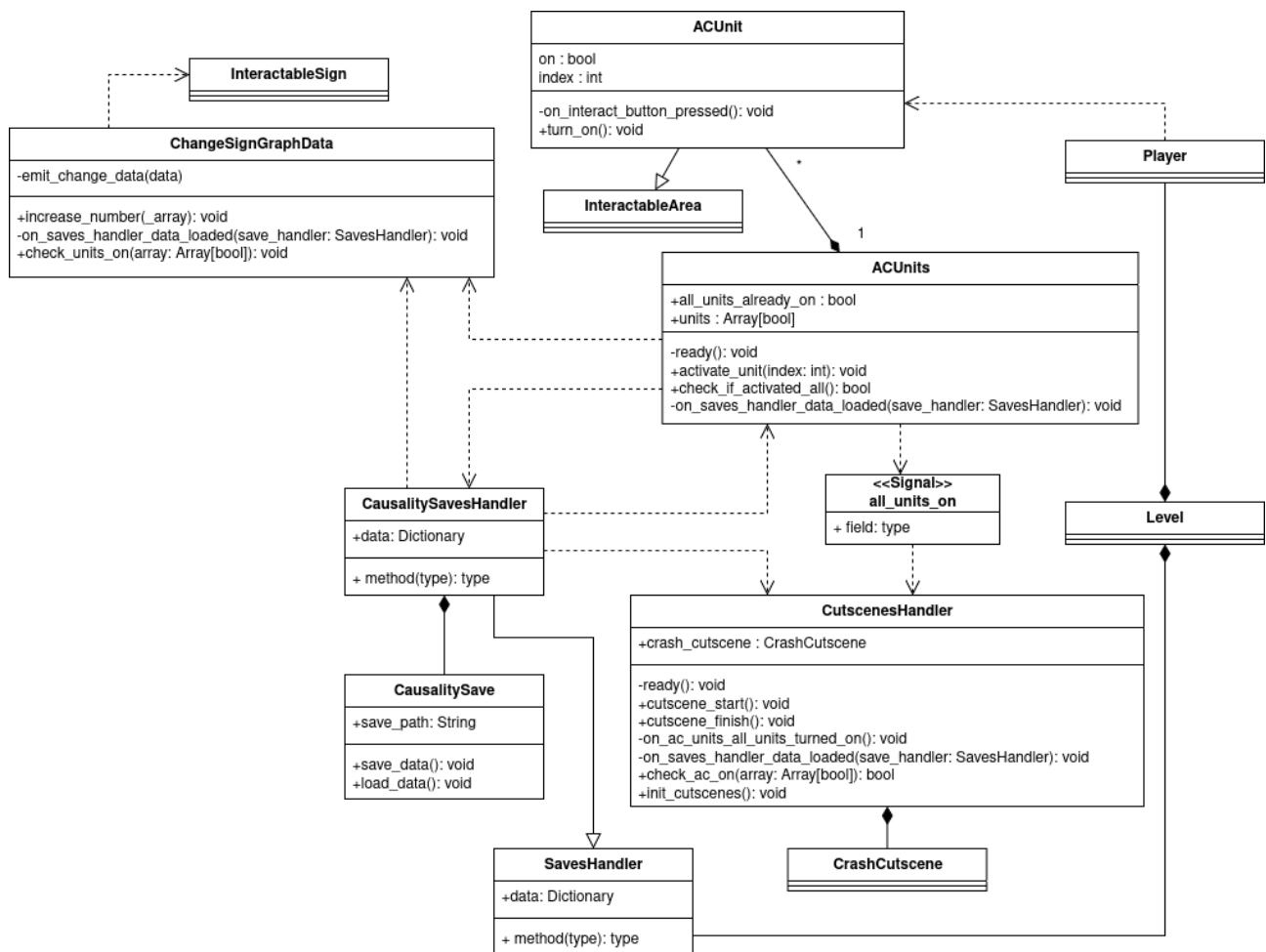


Figura 10: Diagramma del livello della causalità

4.6.1. Scena di intermezzo

5. Requisiti soddisfatti

5.1. Tabella requisiti soddisfatti

5.2. Grafico requisiti soddisfatti